College of Computer and Information Sciences
Computer Science Department

"Meta-Optimization of Genetic Algorithm Parameters for Solving the Traveling Salesman Problem"

CSC 496 – final report

Prepared by:

Lama Almoghamis	443201011
Jumana Alothman	443202103
Tasnim Mohammed Kamal	443204617
Lojien Alabo Omar	444200080
Nouf Al-qahtani	444200409

Supervised by:

Mona Almofarreh

Research project for the degree of bachelor's in computer science

First Semester 1447

I.English Abstract

The Traveling Salesman Problem (TSP) is a well-known NP-hard problem used to find the shortest possible route that visits each node exactly once and returns to the starting point. Genetic Algorithms (GAs) are evolutionary search methods inspired by natural selection, commonly used to solve complex optimization problems. However, the performance of GAs is highly sensitive to hyperparameters such as population size, crossover rate, and mutation rate. Selecting appropriate values for these parameters is a challenging task, as their interactions often affect convergence speed, solution quality, and computational efficiency. Typical methods, such as manual tuning or relying on default parameter values, frequently lead to suboptimal and inconsistent results across problem instances.

For this reason, finding better parameter settings has become important. Automated methods offer a more efficient and consistent option compared to manual tuning. Parameter Meta-Optimization (PMO) helps address this problem by using optimization techniques to automatically find suitable parameter values. PMO can be applied in two ways: offline, before running the algorithm, or online, during the run while adapting to changes.

In this research, we aim to combine both advantages by using meta-learning for offline optimization and an Adaptive Genetic Algorithm for online adjustment. Meta-learning helps the model learn from previous experiences to start with better initial parameters, while the adaptive GA dynamically adjusts its parameters during the search to improve stability and efficiency. Although GA parameter tuning has been widely studied, only a few studies have explored combining meta-learning with Adaptive Genetic Algorithms for solving TSP.

To address this gap, this research uses meta-learning together with an Adaptive Genetic Algorithm to perform Hyperparameter Optimization (HPO) for GA in solving the TSP, using a Neural Network (NN) as the offline prediction model.

II. Arabic Abstract

تُعد مشكلة البائع المتجول (TSP) من المشكلات المشهورة في مجال البحث والتحسين، وتُعد من المشكلات المعروفة بكونها من نوع NP-hard، أي أنها مشكلات صعبة ولا يمكن إيجاد حل مثالي لها بسرعة عند زيادة حجم البيانات. تهدف مشكلة البائع المتجول إلى إيجاد أقصر مسار ممكن يمر بعدة نقاط بحيث يزور كل نقطة مرة واحدة فقط ثم يعود إلى نقطة البداية. تُعد الخوارزميات الجينية (GA) من الطرق التطورية المستوحاة من عملية الانتقاء الطبيعي، وتُستخدم بشكل واسع لحل المشكلات المعقدة.

وقد تم استخدام الخوارزميات الجينية بكثرة لإيجاد حلول فعالة لمشكلة البائع المتجول، إلا أن أداءها يتأثر بشكل كبير بعدة عوامل مثل حجم المجتمع، ومعدل التزاوج، ومعدل الطفرات. ويُعد اختيار القيم المناسبة لهذه العوامل أمرًا صعبًا وبالغ الأهمية، لكونها تؤثر على سرعة الوصول للحل وجودته وكفاءته. وغالبًا ما تؤدي الطرق التقليدية مثل الضبط اليدوي أو استخدام القيم الافتراضية إلى نتائج غير دقيقة أو غير مستقرة بين الحالات المختلفة للمشكلة. لذلك، يهدف هذا البحث إلى استخدام أسلوب يسمى التحسين الفوقي للمعاملات (Parameter Meta-Optimization) من أجل أتمتة عملية البحث عن أفضل القيم الممكنة لمعاملات الخوارزمية الجينية لحل مشكلة البائع المتجول (TSP).

يمكن تنفيذ هذا الأسلوب بطريقتين رئيسيتين، هما التحسين المسبق (Offline) الذي يتم قبل تشغيل الخوارزمية من خلال تحليل البيانات أو التجارب السابقة لتحديد الإعدادات المثلى، والتحسين أثناء التشغيل (Online) الذي يتم أثناء تنفيذ الخوارزمية نفسها مما يسمح بتعديل القيم بشكل تكيّفي حسب الأداء الحالي.

في هذا البحث، نهدف إلى الجمع بين ميزات الطريقتين من خلال استخدام التعلّم الفوقي (Meta-learning) للتحسين المسبق، والخوارزمية الجينية التكيفية (Adaptive GA) للتحسين أثناء التشغيل. يساعد التعلّم الفوقي النموذج على الاستفادة من الخبرات السابقة لتوليد قيم أولية أفضل، بينما تعمل الخوارزمية التكيفية على تعديل هذه القيم باستمرار لتحسين الأداء والاستقرار أثناء التنفيذ.

ورغم الاهتمام المتزايد في هذا المجال، لا تزال الدراسات التي تجمع بين التعلّم الفوقي والخوارزميات الجينية التكيفية محدودة. ولتغطية هذه الفجوة، يقترح هذا البحث استخدام أسلوب التعلّم الفوقي (Meta-learning) بالاقتران مع الخوارزمية الجينية التكيفية (Adaptive GA) لتنفيذ التحسين الفوقي للمعاملات (HPO) للخوارزمية الجينية بهدف حل مشكلة البائع المتجول بكفاءة أعلى، وذلك باستخدام تقنيات مثل الشبكات العصبية (NN) كمثال على أساليب التعلّم الفوقي.

List of Tables

List of Figures

List of Notations

Artificial Bee Colony	ABC
Ant Colony Optimization	ACO
Bayesian Optimization	BO
Biased Random-Key Genetic Algorithm	BRKGA
Crossover Rate	CR
Differential Evolution	DE
Deep Neural Network	DNN
Decreasing High Mutation / Increasing Low Crossover	DHM/ILC
Evolutionary Algorithms	EAs
Evolution Strategies	ES
Fuzzy Evolutionary Algorithm	FEA
Genetic Algorithm	GA
Grouping Genetic Algorithm	GGP
Genetic Programming	GP
Generation	G
Genetic Programming Hyper-heuristic	GPHH
Current generation	g
Harmony Search	HS
Increasing Low Mutation / Decreasing High Crossover	ILM/DHC
Iterated F-Race	iRace

Learned Genetic Algorithm	LGA
Machine Learning	ML
Mutation Rate	MR
Maximum mutation rate (upper bound of MR)	MR_{max}
Maximum Crossover Rate	CR_{max}
Maximum population size (upper bound of PS)	PS_{max}
Minimum Crossover Rate	CR_{min}
Minimum mutation rate (lower bound of MR)	MR_{min}
Minimum population size (lower bound of PS)	PS_{min}
Mean Squared Error	MSE
Neural Network	NN
Parameter Meta-Optimization	PMO
Partially Mapped Crossover	PMX
Partial Optimization Metaheuristic Under Special Intensification Conditions	POPMUSIC
Population Resizing on Fitness Improvement Fuzzy Evolutionary Algorithm	PRoFIFEA
Population Resizing on Fitness Improvement Genetic Algorithm	PRoFIGA
Population Size	PS
Particle Swarm Optimization	PSO
Rectified Linear Unit	ReLU
Rapid Genetic Exploration with Random Direction Hill-Climbing Linear Exploitation	RGHL
Reinforcement Learning	RL
Reduced Variable Neighborhood Descent	RVND
Simulated Annealing	SA
Traveling Salesman Problem	TSP
Training Horizon	TH
Variable Neighborhood Search	VNS

Chapter 1: Introduction

The rapid growth in technology in recent decades has a profound impact on several aspects, including the expansion of communication and trade across the globe. Studies have shown, for example, that e-commerce deliveries now account for more than 20% of total urban transport trips in major cities [58]. Therefore, there is an urgent need to keep pace with this development on various levels, such as finding organized and efficient ways to plan delivery routes, which will help reduce costs and improve service quality.

Hence, the Traveling Salesman Problem (TSP) has gained popularity, it is used to find the shortest possible route that visits each location exactly once and returns to the starting point [1]. Despite its wide applications, in the field of computer science, it is classified as an NP-hard problem, which has prompted many researchers to develop effective solution methods [2].

The Genetic Algorithm (GA) is one of the most widely used heuristic search algorithms for solving the TSP. This algorithm is inspired by human genetics and natural evolutionary processes [3]. However, GA performance can be highly sensitive to parameter values such as population size, crossover rate, and mutation rate. Good parameter settings led to better performance in terms of convergence speed, solution quality, and computational efficiency [4]. Therefore, careful tuning of these parameters is necessary to consistently achieve high-quality results.

The process of determining good parameters is not as trivial as it might seem. One reason is the large number of possible combinations and the fact that some parameters influence how others behave [5]. Tuning parameters based on experience is a common approach, but it requires expert knowledge, which is not always available, especially for non-expert users [6].

Another common approach is to use default parameter values. However, these are often set based on other problems or different contexts, making them suboptimal for the current problem. This inconsistency means that manually adapting parameters for each new instance remain time-consuming and labor-intensive [7]. This has led to the importance of parameter meta-optimization (PMO), which is an automated approach that

uses optimization methods to find the best parameter values for optimization algorithms like GA [8].

A deeper look at PMO reveals a wide variety of methods, including experimental design, metaheuristic algorithms, and trained models, all of which are optimization-based tools. What's striking is that, regardless of the method used, parameter optimization for GA typically occurs either before the run (offline) or during the run (online). Both phases play a fundamental role in the algorithm's performance and effectiveness. This motivated our project to combine both approaches: using a meta-learning protocol for initial (offline) optimization and an adaptive Genetic Algorithm for dynamic (online) adjustment.

1.1 Problem Statement

The TSP instance is represented as a collection of two-dimensional coordinates of cities. Each $City_i$ is defined as a coordinate of x_i and y_i as following:

$City_i = (x_i, y_i)$

Therefor each TSP instance defined as:

$$TSP\ Instance = \{City_1, City_2, \ldots, City_n\} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

TSP is a classical NP-hard problem in which a salesman must visit a set of cities exactly once and return to the starting city while minimizing the total travel distance. The distance D between any two cities is computed using the Euclidean formula:

$$D\ (i, j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$

In the TSP, the problem input is commonly represented by a distance matrix, where each entry dij specifies the distance or cost of traveling from city i to city j. This matrix forms the complete representation of the instance and is essential for evaluating candidate tours as well as extracting statistical features used later in the meta-optimization process.

Because the number of possible routes grows factorially with the number of cities, finding the exact optimal solution becomes computationally impractical for medium and large instances. As a result, Meta-heuristic approaches, such as Genetic Algorithms (GAs). GA has become an essential tool for obtaining high-quality solutions efficiently, such as finding an optimal solution for a TSP. The GA evaluates the quality of each candidate's route using several measures, such as Fitness function, and Gap function. The founded route by the GA is represented as follows:

$$R = [r_1, r_2, \ldots, r_n]$$

Where R is the route founded by the GA, and each element r_i represents the index of a city in the order in which it is visited. Moreover, Fitness function in TSP case is represented as the tour length (the result). Then to compare the GA's result with the known optimal or best-known distance, the Gap function is used. It measures how close the obtained solution is to the optimal one:

Fitness Function:

$$\mathrm{f(R)} = \sum_{i=1}^{n-1}\left(D(r_{\mathrm{i}}, r_{\mathrm{i}}{}^{+1})\right) \; + \; D(r_{\mathrm{n}}, \, r^{1})$$

Gap Function:

$$GAP\ (\%) = \frac{Obtained\ Tour\ Length - Known\ Optimal\ Tour\ Length}{Known\ Optimal\ Tour\ Length} \times 100$$

Although GAs are powerful and flexible, their performance depends heavily on several key parameters such as population size, crossover rate, and mutation rate. Poor parameter settings may lead to slow convergence, premature stagnation, or low-quality solutions. Manual tuning requires expert experience and does not generalize well in different TSP instances. Over the years, many methods have been developed to tune these parameters. Offline parameter optimization techniques use prior experiments, statistical models, or machine learning approaches to predict good parameter values before running the algorithm. Similarly, online parameter optimization techniques update parameters dynamically during the run using adaptive rules, feedback mechanisms, or self-adjusting

strategies. Both offline and online methods have shown strong potential and are widely used in literature. However, only a limited number of studies have combined these two approaches into a single hybrid framework. This creates a possibility to explore hybrid methods that combine offline and online methods.

1.2 Goals and Objectives

The goal of this project is to enhance the performance of GAs in solving the TSP by applying a combination of both PMO methods (offline and online approaches). Therefore, this study investigates the following research question: Does a hybrid offline–online meta-optimization approach enhance GA performance for the TSP? This study contributes by evaluating this method and demonstrating their impact on solution quality, convergence speed, and computational efficiency.

Objectives

To achieve this goal, the project will:

- Define the problem by outlining the main challenges of applying GAs to the TSP, particularly sensitivity to parameter settings and risk of premature convergence.
- Review related literature to understand existing GA parameter tuning approaches and identify gaps in current research.
- Design the research methodology, including the selection of meta-optimization techniques (e.g., offline meta-learning and online adaptive strategies) and the experimental setup.
- Develop a modular framework to implement, apply, and compare different meta-optimization methods for GA parameter tuning.
- Evaluate performance using standard TSP benchmark instances, comparing methods in terms of solution quality, execution time, and computational efficiency.
- Analyze and interpret results to report key findings, practical implications, and limitations of the study.

1.3 Proposed Solution

To address the challenge of manually tuning GA parameters for the TSP, we propose a Hybrid PMO framework that combines Meta-Learning for offline parameter prediction with an adaptive GA for online parameter adjustment. Specifically, we plan to use a Deep Neural Network model to automatically determine effective GA parameter settings (e.g., population size, crossover rate, mutation rate) based on TSP instance features and allow the GA to dynamically adapt these parameters during execution. This approach aims to improve solution quality and convergence speed without relying on manual tuning or expert knowledge.

1.4 Research Scope

This research focuses on meta-optimization of GA parameters specifically for solving the TSP. The primary objective is to develop strategies for automatically configuring or adapting GA control parameters, such as population size, crossover rate, mutation rate—both before the algorithm begins (offline) and during its execution (online). The evaluation will include moderate-scale TSP instances (e.g., <150 cities) that will not require a high-performance computing resource.

1.5 Research Significance

This research contributes to the field of combinatorial optimization by developing a hybrid Parameter Meta-Optimization (PMO) framework that automatically tunes Genetic Algorithm (GA) parameters for the Traveling Salesman Problem (TSP).

Since TSP is a fundamental model for many real-world applications-such as logistics routing, delivery planning, network design, and manufacturing scheduling-improving the way it is solved can lead to significant time and cost savings in practice.

Manual parameter tuning is not only time-consuming but also requires expert knowledge, which is often unavailable. By combining Meta-Learning (e.g., Deep Neural

Networks) for offline configuration with an adaptive GA for online adjustment, our approach reduces human effort while improving solution quality and convergence speed.

This work is therefore worth performing because it offers a practical, automated, and efficient way to enhance GA performance on TSP and similar NP-hard problems-making optimization more accessible and effective for real-world users.

1.6 Ethical and Social Implications

Although the connection with ethical and social issues might seem indirect, improving the solution of the TSP could potentially enhance the performance of many real-world systems that depend on delivery, scheduling, and transportation. These systems rely on large amounts of user and operational data, which makes issues of data handling and privacy important. Better optimization can reduce the need for collecting too much data, lower the risk of exposing sensitive information, and support safer decision-making in real-world applications.

1.7 Report Organization

This report is organized as follows. Chapter 2, Background, presents foundational information, including an overview of the Traveling Salesman Problem (TSP), optimization algorithms, Genetic Algorithms (GA), and Parameter Meta-Optimization (PMO). Chapter 3, Literature Review, reviews related studies and discusses previous PMO approaches, including offline, online, and hybrid methods, followed by a summary. Chapter 4, Methodology first presents the problem representation, then introduces the framework overview, followed by the neural network offline phase and the online adaptive phase. Chapter 5, Experimental Design, outlines the datasets used, experimental parameters, evaluation metrics, and the stated hypothesis. Chapter 6, Conclusion summarizes the work and presents the main findings achieved so far.

1.8 Summary

Chapter 1 provides an overview of the motivation and foundation of the study. It begins by highlighting the rapid growth in technology and the increasing demand for efficient route-planning systems, such as those used in delivery and transportation services. The chapter explains that the TSP has become a central model for solving such routing challenges, as it aims to find the shortest possible route that visits each city once and returns to the starting point. Because the TSP is an NP-hard problem, many researchers use heuristic and meta-heuristic methods to obtain high-quality solutions efficiently.

Among these methods, the GA is widely used; however, its performance is highly sensitive to parameter choices such as population size, crossover rate, and mutation rate. Selecting good parameters is difficult because these values interact with one another and default settings often do not perform well for new TSP instances. Manual tuning also requires expert knowledge and is time-consuming, especially for non-expert users.

To address these challenges, the chapter introduces Parameter Meta-Optimization (PMO), an automated approach that seeks the best parameter values for optimization algorithms. PMO methods can operate offline, before the algorithm runs, or online, during execution. Since both stages play important roles, the project proposes combining them through a hybrid approach. Specifically, the study suggests using meta-learning (via a neural network) to predict initial parameter values offline, together with an adaptive GA to adjust parameters dynamically during execution.

The rest of the chapter presents the problem statement, project objectives, the proposed solution, research scope, and the significance of the study. It emphasizes that a hybrid PMO framework can reduce human effort and improve GA performance, making optimization more accessible for real-world applications such as logistics routing, network design, and scheduling systems.

Chapter 2: Background:

This chapter provides the essential background required to understand the methods and concepts used throughout this study. It introduces the Traveling Salesman Problem (TSP), its mathematical formulation, and key characteristics relevant to the optimization process. The chapter also outlines the fundamental principles of Genetic Algorithms (GAs), which form the basis of the proposed hybrid meta-optimization approach.

2.1 Traveling Salesman Problem (TSP)

The Traveling Salesman Problem (TSP) is a combinatorial optimization problem that has been studied in the field of combinatorial optimization since the 1930s [10]. The problem is used to find the shortest possible route that visits a group of cities (nodes) exactly once and returns to the starting point [11][35]. The TSP is classified as an NP-hard problem [37], which means that there is not any algorithm that can solve it efficiently in polynomial time [11]. For this reason of its difficulty and importance, it has been studied in several fields such as mathematics, computer science, and operations research [13],A general flowchart of the TSP solution process is illustrated in Figure 1.

Formally, the Traveling Salesman Problem (TSP) can be described mathematically. Let $G = (V, E)$ be a graph, which can be directed or undirected, and let F be the set of all Hamiltonian cycles (tours) in G. Each edge e in E has a cost or weight c_e [35] [13]. The Traveling Salesman Problem is to find a Hamiltonian cycle in G such that the total sum of the edge costs is as small as possible [35] [36], An example of a weighted graph representation of a TSP instance is shown in Figure 2.

The Traveling Salesman Problem (TSP) has many practical uses. One use is to plan routes for vehicles in the most efficient way [13]. Another use is in computer wiring, where it helps connect components properly [38]. TSP can also be applied to cutting materials like wallpaper and to arranging jobs in order [37] [13]. In scientific research, it is used in X-ray crystallography to plan sequences more efficiently and reduce effort [11].

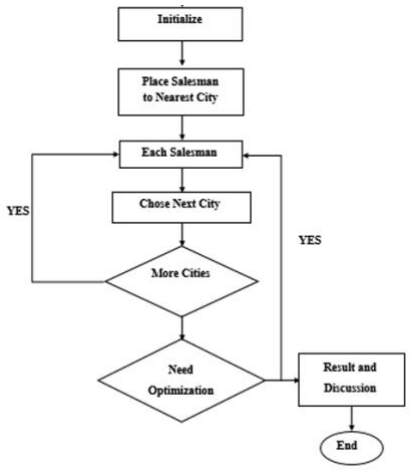

Fig. 1. General flowchart of the (TSP) solution process [13].

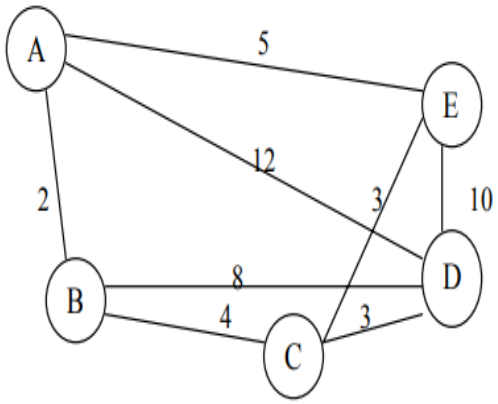

Fig. 2. Weighted graph representation of the Traveling Salesman Problem [20]

In practice, TSP instances fall into two main categories: symmetric and asymmetric. In the symmetric TSP (STSP), the distance between two cities is the same in both directions, meaning that $d(i,j) = d(j,i)$. This situation is common in Euclidean problems where distances are based on geometric coordinates. In contrast, the asymmetric TSP (ATSP) allows different costs for traveling from city i to city j and from city j to city i, that is $d(i,j) \neq d(j,i)$. Such cases typically arise in real-world routing problems involving one-way streets, traffic conditions, or direction-dependent travel constraints. Because this distinction affects the structure of the distance matrix, it is included as an input feature in the neural network used for parameter prediction.

2.2 Optimization Algorithms

Optimization algorithms are methods used to find the best solution to a problem by either minimizing or maximizing a specific goal (called an objective function) [16]. They are widely used in many areas, including engineering, computer science, and operations research, to solve practical problems such as scheduling, resource allocation, route optimization, and TSP [17]. Generally, optimization algorithms fall into two main groups: Exact Algorithms and Approximate Algorithms [18].

It is important to note that there is no single agreed-upon way to classify optimization algorithms-different researchers use different classifications based on their focus. For example, Fig. 3 and Fig. 4 show two different ways that researchers have used to classify these algorithms.

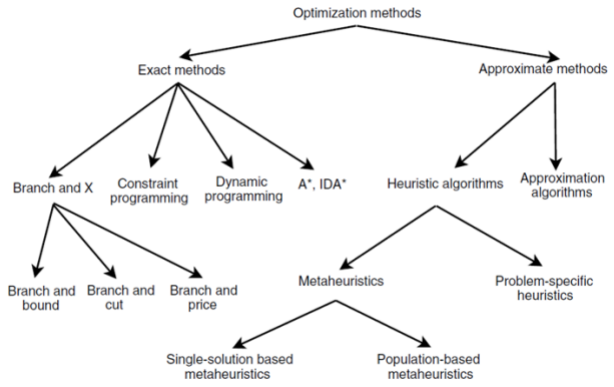

Fig. 3. Branches ranch of Optimization methods [24].

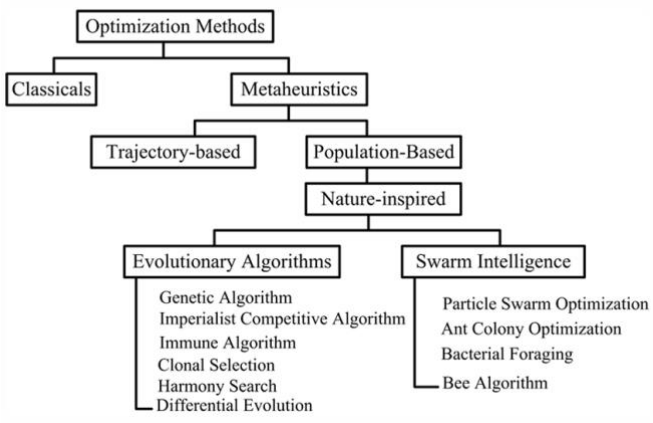

Fig. 4. Branches of Metaheuristic [25].

We chose a classification that fits our research goals. As shown in Fig. 5, we organize optimization algorithms into Exact and Approximate methods, and further divide Approximate algorithms into Heuristics and Metaheuristics, with Metaheuristics split into four clear types: Evolutionary, Swarm-Based, Physics-Based, and Al-Based. This structure supports our focus on parameter tuning and meta-optimization [43],[44].

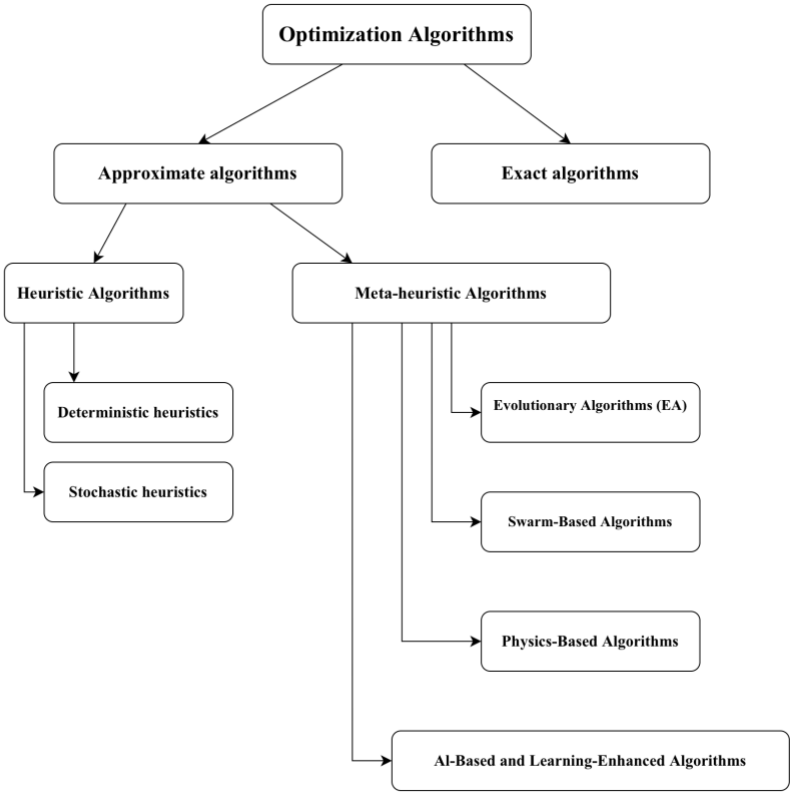

Fig. 5. The branches of Optimization Algorithms.

Exact algorithms are designed to find the guaranteed optimal solution by systematically exploring the entire search space [17]. While they give perfect results, they often require a lot of time and computing power, especially for large or complex problems like TSP, which belongs to the class of NP-hard problems [17]. Common examples include Dynamic Programming, Branch and Bound, and Integer Linear

Programming-methods that work well for small instances but become too slow as the problem size grows [17].

Approximate algorithms aim to find good-quality (near-optimal) solutions quickly, making them more practical for real-world problems where time and resources are limited [18]. They are especially useful for large scale or complex problems where exact methods are too slow [18]. These methods are generally divided into two main types: heuristics, which are simple, problem specific rules, and metaheuristics, which are high-level strategies that guide heuristics to explore the search space more effectively and avoid local optima [18].

2.2.1 Heuristic Algorithms

Heuristic algorithms are simple and fast methods that try to find a good solution to a problem without checking all possible options [20]. They don't always find the best answer, but they give a reasonable one in a short time. Their results depend a lot on the type of problem they are used for [33]. Heuristics includes two main branches: Deterministic heuristics and Stochastic heuristics [7].

Deterministic heuristics always follow the same steps and give the same result for the same problem [17]. For example, in TSP, the Greedy algorithm (like Nearest Neighbor) starts at a city and keeps going to the nearest unvisited city until the tour is complete [17]. Stochastic heuristics use some randomness during the search, so they can give different results each time [9]. An example is a randomized version of insertion methods used to build TSP tours [33]. In short, even though heuristics are quick, they often get stuck with a weak solution and can't easily find better ones [18].

2.2.2 Meta-heuristic Algorithms

Meta-heuristic algorithms are high-level strategies that guide simpler heuristics to explore the search space more effectively and avoid getting stuck in local optima [18]. They are inspired by natural, biological, or physical processes and can be applied to many different problems such as TSP. It does not need to change the structure, or it might require a bit [25]. Because TSP is NP-hard, meta-heuristics are among the most popular approaches to find good solutions in reasonable time [18]. Meta-heuristics are commonly grouped into the following categories: Evolutionary Algorithms (EAs), Swarm-Based Algorithms, Physics-Based Algorithms, Al-Based and Learning-Enhanced Algorithms [18] [43] [44].

In practice, the performance of these algorithms (such as GA) depends heavily on their settings, such as mutation rate or population size [43]. To avoid guessing these values by hand, researchers now use meta-optimization which means using one optimizer to automatically tune the parameters of another [44].

A) Evolutionary Algorithms (EA)

Evolutionary algorithms (EAs) are inspired by the process of natural evolution, where a population of candidate solutions improves over generations using operators like selection, crossover, and mutation [18]. These algorithms are especially useful for complex problems because they explore many possible solutions at the same time [3]. The most well-known EA is the Genetic Algorithm (GA), but other types include Evolution Strategies (ES), Genetic Programming (GP), and Differential Evolution (DE) [15]. Among them, GA is the most used for TSP due to its strong ability to handle large and discrete search spaces [13].

B) Swarm-Based Algorithms

Swarm-based algorithms mimic the collective behavior of social animals, such as ants, birds, or bees, where simple agents cooperate to solve complex tasks [25]. Well-known examples include Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO), and Artificial Bee Colony (ABC) [25].

C) Physics-Based Algorithms

Physics-based algorithms are inspired by physical laws or natural phenomena to guide the search process [18]. A classic example is Simulated Annealing (SA), which mimics the cooling of metals to gradually find better solutions. Another is Harmony Search (HS), which imitates how musicians improvise to find the best harmony [25].

D) Al-Based and Learning-Enhanced Algorithms

Recent approaches combine meta-heuristics with artificial intelligence to make the search process adaptive [43]. For example, some methods use learning techniques to automatically adjust GA parameters during the search, reducing the need for manual tuning [43]. These strategies fall under the broader concept of meta-optimization, where one optimizer is used to improve another [22]. Common examples include Reinforcement Learning (RL), Q-Learning, and Neural Network Optimization [22].

2.3 Genetic Algorithms

2.3.1 GAs definition

As we mentioned in the previous section 2.2, Genetic Algorithms (GAs) are one of the EAs that belong to the Meta-heuristic family, which is part of the Optimization Algorithms [18]. At the 1960s Genetic Algorithms GAs was proposed for the first time by John Holland [3]. Later, in the University of Michigan, Holland and his students and colleagues further developed this algorithm [3]. GAs are a powerful algorithm that archives high-quality search and optimization [19] [21]. This algorithm is inspired by the natural selection of genes [3] [19]. Moreover, GAs has parameters (population size, crossover rate, and mutation rate) that serve an important role in affecting the performance of the algorithm [19]. in a simple GA parameters are changed based on three main operations selection, crossover, and mutation [15] [19]. GAs are used in many practical applications and problem solving like TSP, scheduling problems, and more [21].

2.3.2 Implementation

Simple GA is implemented based on the general concept of GAs, which is not for a specific problem. As shown in Fig. 6, the algorithm flow starts by creating a random population of chromosomes (often represented as a string). Then then works throw the next steps:

1) **Start**: creating a random population of chromosomes. Generation should include a wide variety of genes to help the algorithm explore a large space of possible solutions and find the optimal solution.
2) **Evaluation**: measuring how well a chromosome performs or how suitable the solution is by applying the fitness function. In optimization problem, the parameter goal is to maximize the fitness function value for a better solution [21].
3) **Searching for a new population**: This loop starts by selecting the best parents by selection operation. Then, parent genes are mixed to create a new generation (children) by crossover operation. After crossover, each offspring undergoes mutation with a probability (mutation rate), regardless of whether crossover

occurred, or whether the solution is good or bad. Mutation introduces random changes to maintain genetic diversity and help the algorithm explore new regions of the search space.

4) **New population replacement**: Last step before ending the run, the loop replaces the old population with the new generation, for the next round of the algorithm.

These steps form the core iterative cycle of the genetic algorithm and are based on established GA principles [3], [15], [19].

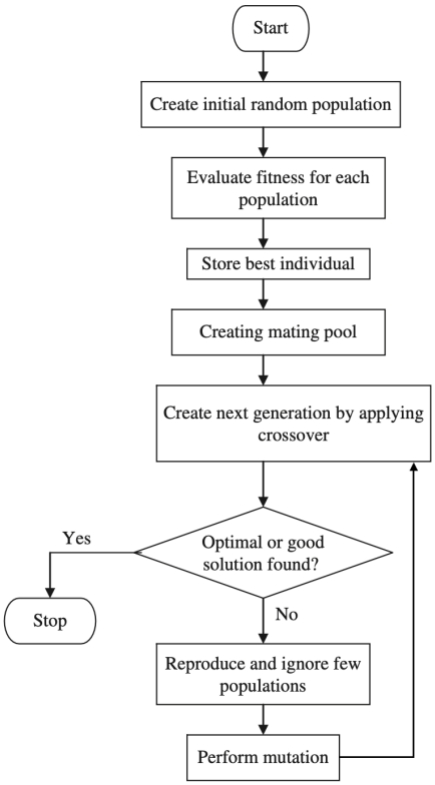

Fig. 6. Flowchart of a genetic algorithm [15].

2.3.3 Genetic Selected Operators for TSP

The GA generates the new generation using a set of genetic operators [59]. The selected operators in this paper are explained as follows:

• Tournament selection:

Choose a fixed number of individuals randomly (tournament size), then pick the one with the highest fitness among them to be the parent [48]. []. For example, in a tournament of size 3, three individuals are chosen randomly from the population, and the one with the highest fitness is selected as a parent.

• Partially Mapped Crossover (PMX):

PMX starts by choosing two crossover points randomly, and the cities between these points are copied from Parent A to the child. Then, each city in the copied segment is mapped to its corresponding city in Parent B to form a mapping relationship. After that, the remaining empty positions in the child are filled as follows: if the city from Parent B does not appear in the copied segment, it is placed directly in the same index; otherwise, the mapped value is used to avoid duplication [60].

Example:
Parent A: [4 6 2 3 5 1]
Parent B: [6 3 2 1 5 4]
Selected crossover points: index 2 to index 4

Copied segment from Parent A:
Child: x x 2 3 5 x x

Mapping (Parent A → Parent B):
2 → 2, 3 → 1, 5 → 5

Filling the remaining positions:
Index 0: 6 not in segment → place 6
Index 1: 3 in segment → mapped to 1
Index 5: 4 not in segment → place 4

Resulting offspring:
[6 1 2 3 5 4]

• Inversion mutation:
Two points are selected randomly in the route, and the segment between them is reversed [60].

Example:
Before: [6 1 2 3 5 4]

Selected points: index 1 and index 3
After inversion: [6 3 2 1 5 4]

2.3.4 Challenges

Genetic Algorithms face several challenges in optimization especially Traditional GA, we will mention some of them:

- **Non-stationary search dynamics:**

This term refers to the crucial challenge in evolutionary optimization where the effectiveness of genetic operators, such as crossover and mutation, is not fixed but undergoes dynamic shifts throughout the evolutionary search process, this phenomenon makes the adaptation of traditional operator selection strategies very hard. [29]

- **Computational Cost:**

GAs typically needs many response (fitness) function evaluations. The time needed for a GA to converge is proportional to O(nlogn) function evaluations, where n is the population size. This means that using a larger population increases the computational cost, memory use, and overall running time. [15]

- **Local Optima and Premature Convergence:**

It is a common problem in GA which occurs when the population settles too early on a solution that isn't the best, leaving not enough diversity to keep improving [19][15].

2.4 Parameter Meta-Optimization

Meta-Optimization is an important approach for solving optimization problems because it aims to find the best parameter values for optimization algorithms, especially GAs in this project [27]. Meta-Optimization improves the performance of an optimizer by making optimization work better [14]. It achieves this by combining the results of several different optimization methods in a systematic and informed way [14].

Meta-heuristic algorithms are strongly affected by the values of their parameters, since these parameters directly control the heuristic mechanisms of the algorithm [9]. Incorrect parameter settings can cause the algorithm to behave incorrectly, get stuck in a local optimum, or move randomly without reaching a solution within the allowed time [26]. Determining optimal parameter values is a difficult task [8]. This difficulty appears because the objective function is nonlinear, the parameters interact with each other, and there are many local optima [8]. Furthermore, evolutionary algorithms do not have exact mathematical methods to compute their best parameter values [5].

Parameter Meta-Optimization (PMO) was therefore established to address these challenges [9]. PMO is not a new concept; it originated in the late 1970s when *Mercer and Sampson* first applied it to optimize evolutionary algorithms [9][8]. PMO automates the selection of optimal parameter values for a given algorithm [14][26]. This approach treats parameter selection as an optimization problem within the parameter space rather than the solution space [9].

PMO can be implemented by various methods, such as using other optimization algorithms, mathematical techniques, or ML models such as Neural Networks (NN) [9] [52]. ANN machine is a group of connected processing units, often called neurons or nodes, inspired by biological neurons. Its ability to process information (its knowledge) does not come from explicit programming. Instead, it is stored in the connection of weights between the nodes. These weights are learned and adjusted through training on an example dataset.

2.5 Neural Networks

A neural network (NN) is a computational model composed of interconnected processing units called neurons, inspired by the structure of the human brain. These neurons are typically organized into layers: an input layer that receives data, one or more hidden layers that process information, and an output layer that produces the final prediction. Each connection between neurons carries a weight, and each neuron applies a bias before passing the weighted sum through a nonlinear activation function to generate its output [54].

Neural networks are widely used in machine learning for tasks such as classification, regression, and pattern recognition. Their ability to approximate complex nonlinear functions makes them suitable for modeling relationships between inputs and continuous or discrete outputs. In regression tasks—where the goal is to predict numerical values, the network is trained to minimize a loss function such as Mean Squared Error (MSE), which quantifies the average squared difference between predicted and actual target values [55].

A common choice for the activation function in hidden layers is the Rectified Linear Unit (ReLU), defined as:

$$ReLU(x) = max(0, x)$$

ReLU is favored for its simplicity, computational efficiency, and effectiveness in mitigating the vanishing gradient problem during training, which helps multilayer networks converge more reliably [56].

Even shallow feedforward networks with a few hidden layers can capture meaningful patterns from structured input data. Because of this modeling capacity and their relatively straightforward training process, neural networks are frequently employed as predictive models in data-driven optimization and meta-learning contexts [57].

2.6 Summary

Chapter 2 provides the foundational concepts needed to understand the literature review and methodology. It begins with a detailed explanation of the TSP, its mathematical formulation, and its practical applications in routing, manufacturing, and scientific planning. The chapter distinguishes between symmetric and asymmetric instances and explains why TSP is a benchmark problem for testing optimization algorithms.

Next, the chapter reviews optimization algorithms, dividing them into exact and approximate methods. Exact methods guarantee optimal solutions but are computationally expensive, while approximate methods, including heuristics and meta-heuristics, trade optimality for speed and scalability. Meta-heuristics such as Evolutionary Algorithms, Swarm-based algorithms, Physics-based methods, and AI-enhanced algorithms are introduced, highlighting their flexibility and problem-independent search strategies.

A more focused discussion of Genetic Algorithms (GAs) follows, describing their inspiration from natural evolution, their key steps (selection, crossover, mutation), and the challenges they face, such as premature convergence, computational cost, and sensitivity to parameter settings.

Finally, the chapter explains Parameter Meta-Optimization (PMO) as a structured way to automatically select good parameter values using techniques ranging from experimental design and surrogate models to machine-learning-based approaches. The chapter also introduces Neural Networks (NNs) as predictive models capable of learning the relationship between problem features and optimal parameter settings. This background establishes the foundation for the hybrid offline–online PMO approach proposed in later chapters.

Chapter 3: Literature Review

In this chapter, we reviewed the methods used by related studies on PMO. While examining related studies, we observed that there are many different techniques for PMO. These techniques can be grouped according to the time when PMO takes place. Either before execution (offline) or during execution (online). Furthermore, some studies combined both approaches. Therefore, we introduced a hybrid approach.

3.1 Offline PMO

Offline PMO is a statistical method used to find the best parameter settings for an algorithm before running it. It usually needs many runs of the algorithm to test its performance on one or several problem instances with different parameter settings. This makes the process slow and time-consuming, which is its main disadvantage. However, its main advantage is flexibility, since a good tuning method can be used for many different meta-heuristic algorithms [9]. There are several ways to tune parameters statically of an algorithm, for EAs especially GA. Common methods for this include Meta-EAs, Design of Experiments (DOE), Surrogate-based, Meta-Learning, and other algorithms [39] [32] [45] [28] [34] [47].

3.1.1 Meta-EAs

Mata-EAs is a Meta-heuristic optimization algorithm in the (Meta-level algorithm) that focuses on finding the best parameter values for a given Optimization Algorithm in the (Base-level Algorithm) that solves a specific problem in the (Base-level problem) as shown in Fig. 7 [8] [9]. Therefore, EA can be used as a PMO for any other EA. Meta-EAs are also called heuristic search-based, most Meta-EAs have two common characteristics: the use of stochastic operators and the balance between exploration and exploitation [39].

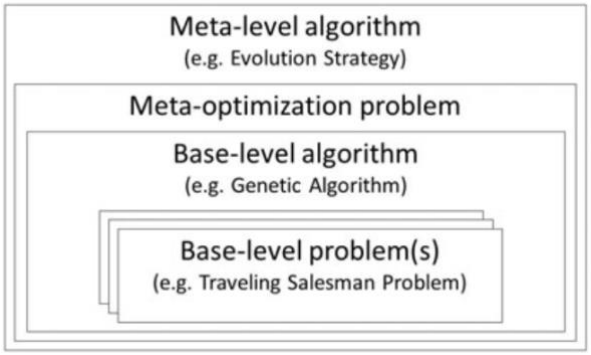

Fig. 7. Meta-EAs concept [8].

According to [42], The Meta-GA is an offline tuning approach that employs one genetic algorithm to optimize the parameter settings of another, including parameters such as population size, crossover rate, and selection method. Thay investigated the challenge of hyper-parameter tuning in test case generation, a process that can be computationally expensive and not always beneficial for all software classes. They proposed a strategy called “Tuning Gain”, which estimates the potential benefit of tuning for each class using source code metrics, allowing the process to target only the most promising classes [42]. The results demonstrated that allocating the tuning budget exclusively to these high-potential classes performed significantly better than using global or default configurations. This approach enhanced branch coverage and improved the overall efficiency of the tuning process by saving time and focusing on computational resources where they were most effective [42].

3.1.2 Design of Experiments

The Design of Experiments (DOE) is a structured framework (as represented in Figure 8) used to analyze and evaluate all possible combinations of parameter values for an experiment [31] [32] [33].

The paper [33] applied DOE to determine the most effective parameter settings for a GA in solving a real-world delivery route problem related to the TSP [33]. They conducted statistical tests to identify which parameters had the strongest effect on route distance [33]. The results indicated that all parameters, as well as several of their interactions, significantly influenced performance [33]. The optimal parameters configuration are population size = 90, crossover rate = 1.00, mutation rate = 0.010, and 800 iterations, producing the shortest total path, showing improved results over other tested combinations [33]. The study concluded that considering parameter interactions leads to better results than adjusting each parameter separately or relying on intuition [33].

Later, according to [32], they also used DOE with GA to optimize the placement of turbines in a wind farm, aiming to maximize energy output while minimizing costs. Their findings showed that both crossover and mutation rates had a strong influence on performance, particularly when these parameters interacted. The optimal configuration achieved higher energy production, improved efficiency, and enabled more effective turbine placement within the same area. The study highlighted that relying on trial and error is suboptimal, and that a structured approach like DOE significantly improves the speed and quality of the optimization process.

3.1.3 Surrogate-based

According to [44] and [45], A surrogate model is a simplified computational tool used in computationally heavy tasks such as optimization and analysis to reduce computational cost and complexity. In surrogate-based tuning, the model learns to predict the performance impact of different hyper-parameter settings, enabling efficient exploration of promising regions without relying on probabilistic models or acquisition functions.

Based on [44], Bayesian Optimization (BO) is an offline parameter tuning approach that constructs a probabilistic model to estimate how different parameter settings influence an algorithm's performance. It iteratively evaluates new configurations, balancing exploration of new options with optimization of known high-performing settings. Thay applied BO to tune the parameters of a Grouping Genetic Algorithm (GGA) for a complex routing problem. Their study aimed to automatically identify effective parameter settings that improve algorithm performance across different problem instances. The results showed that BO improved the performance of the algorithm compared to manual and random tuning methods, confirming its effectiveness and efficiency for tuning metaheuristic algorithms in complex routing problems. [44]

Following the study of *Kim and Joe* (2024) proposed the Rapid Genetic Exploration with Random Direction Hill-Climbing Linear Exploitation (RGHL) method, a hybrid approach addressing the imbalance between exploration and exploitation in traditional optimization methods such as Bayesian optimization. Their aim was to develop a more stable and balanced strategy that improves performance and avoids convergence to local optima. The study showed that RGHL outperformed several existing methods and achieved faster, more reliable convergence, though at the cost of increased computational time. [45].

3.1.4 Meta-Learning

Meta-learning is a branch of machine learning that focuses on parameter and algorithm selection [28]. It learns from the past performance of algorithms to predict or recommend the most suitable algorithm for a new problem based on its characteristics [28] [41].

In the paper [28], they used Meta-learning before the optimization process begins. For instance, they employed Meta-Learning to select the best Meta-heuristic (MH) for a given TSP instance. Their method builds a predictive model that maps specific characteristics of a TSP instance (called meta-features) such as graph structure, symmetry, or connectivity to the expected performance of various meta-heuristics (e.g., Genetic Algorithms, Ant Colony Optimization, Tabu Search). This allows the system to recommend the most suitable algorithm before execution, leading to better solutions without having to test all methods [28].

Another study by [46] they discovered a new genetic algorithm using meta-optimization. It replaces traditional operators with attention-based neural modules-for example, the way parents are selected or how mutation rates are adjusted is handled by a learned neural network. These modules are trained offline on a set of optimization problems, and the final algorithm (called LGA) is then applied to new problems. Because the entire logic of core GA components is learned and replaced, this is a good example of algorithm structure optimization: the architecture itself is the output of the meta-optimization process. [46]

Another study uses Genetic Programming Hyper-heuristics (GPHH) as a learning method that uses GP to automatically create new heuristic parts, such as selection operators, for EAs. They study focuses on the problem that GA performance depends heavily on manually chosen selection operators, which may not work well for all problems. The goal was to use GPHH to automatically build a selection operator that performs well on different optimization problems, including new ones not seen before.

The results showed that the new operator created by GPHH performed as well as or better than standard methods in many test problems. It worked better than proportional selection and was competitive with ranking-based approaches. Overall, the study proved that GPHH can successfully and automatically design effective and adaptable components for GAs. [41]

3.1.5 Other used algorithms

A) Iterated F-Race Algorithm

According to [34], the Iterated F-Race (iF-Race) algorithm is an automated parameter tuning method that employs a statistical procedure to evaluate multiple parameter configurations across different problem instances. It progressively eliminates poorly performing configurations while maintaining those that show better performance [34]. They applied the iF-Race algorithm to a GP-system to explore its hyper-parameter space and identify parameter settings that enhance predictive accuracy while maintaining a compact model size. The results indicated that iF-Race effectively identified configurations that matched or slightly outperformed manually tuned settings in terms of test error. The study concluded that optimizing parameters only for predictive accuracy is suboptimal for GP-systems where model size and explainability are also essential [34].

B) HORA method

As described by [47], the Heuristic Oriented Racing Algorithm (HORA) is an offline parameter tuning method that integrates Design of Experiments (DOE) with racing algorithms to efficiently explore a limited hyper-parameter space. The process begins with DOE applied to a small training set to identify promising parameter ranges. It then refines the search by testing new configurations near the best-performing ones through a racing procedure that eliminates weaker parameter sets based on statistical analysis. [47]

This paper [47] addressed the challenge that Meta-heuristics such as SA and GA are highly sensitive to parameter settings. Their objective was to develop a faster and more effective tuning method that reduces computational cost while maintaining or improving solution quality for optimization problems such as the TSP. [47]

The results demonstrated that HORA significantly reduced tuning time compared to conventional racing algorithms, while maintaining or slightly improving performance. By concentrating on the search around promising ranges rather than evaluating all possible configurations, HORA efficiently identified high-quality parameter settings. The study concluded that combining DOE with the racing algorithm makes parameter tuning more applicable, efficient, and scalable for Meta-heuristic algorithms. [47]

3.2 Online PMO

Another approach of PMO is parameter control (also known as on-line tuning), which considers dynamically adjusting values of parameters during the optimization problem execution or run via appropriate strategies [9]. In general, papers that focus on online meta-optimization techniques address a fundamental challenge in evolutionary algorithms: non-stationary search dynamics—the fact that the effectiveness of genetic operators (like crossover and mutation) changes over time, depending on the problem instance, the current population, and the stage of the search.

3.2.1 Deterministic control

Some papers adjust GA parameters using fixed rules or schedules, without considering feedback, which is called “deterministic control” [48][49]. Paper [4] explicitly tested four GA parameters (Generation (G), Population Size (PS), Crossover Rate (CR), and Mutation Rate (MR)) in two phases. The first phase used static tuning, showing positive correlations with fitness and negative correlations with execution time, revealing a trade-off. Overcoming this trade-off was one of the motivations to move to the second phase. G was adaptively controlled, while the other three were adjusted deterministically based on first-phase observations, with limits for each. For example, CR

was stable across 0–100, so it started high to enhance exploration, then decreased gradually according to a specific equation to focus on exploitation. And MR was stable at ≥30%, so it started low, then increased gradually to add diversification later. After the whole experiment, they concluded that applying these four adaptive parameters together yielded optimal results, improving fitness by 1.0%, and execution time by 38.7% compared to the standard static genetic algorithm.

Another paper that proposed new deterministic approaches to control parameters is [48]. It introduced two methods: Increasing Low Mutation / Decreasing High Crossover (ILM/DHC) and Decreasing High Mutation / Increasing Low Crossover (DHM/ILC). ILM/DHC starts with high crossover (100%) and low mutation (0%), then gradually and linearly decreases crossover from 100 to 0 while increasing mutation from 0 to 100. The other approach does the opposite. The study concludes that ILM/DHC is more effective with small population sizes by gradually increasing mutation to enhance diversity, while DHM/ILC is more effective with large population sizes by focusing on increased crossover for exploitation towards the end of the algorithm run. Compared to common static settings (50%–50% mutation–crossover and 0.03MR–0.9CR), these deterministic methods clearly outperformed them in both solution quality and execution speed.

3.2.2 Adaptive control

Many papers instead of depending on fixed or predefined rules, depend on the feedback of the performance to control the parameters, which is known as “adaptive control” [48] [49],. Go back to paper [4], which tested two phases: static and adaptive, and adjusted parameters in the second phase deterministically. One of its parameters was controlled adaptively. If the fitness of the algorithm stays fixed and does not change for a certain number of generations, G will decrease or change based on an adaptive strategy. Another paper that tried to introduce an adaptive approach is [50]: Population Resizing on Fitness Improvement Fuzzy Evolutionary Algorithm (PRoFIFEA). The paper integrates two methods: Fuzzy Evolutionary Algorithm (FEA) to control crossover and mutation rates, since it takes the current population number of generations as input to maintain diversity and avoid early convergence; and it uses Population Resizing on

Fitness Improvement Genetic Algorithm (PRoFIGA) concept, which increases and decreases population size based on the best fitness value achieved in each generation. They tested the model on TSP to find the shortest route between 20 cities. The comparison demonstrated that the PRoFIFEA genetic algorithm produced a better/shorter route and a more optimal final solution than the standard genetic algorithm. PRoFIFEA achieved a fitness value of 0.0069 and a route length of 143.4062 versus a fitness value of 0.0058 and a route length of 171.6663. This confirms that the hybrid technology (PRoFIFEA) produces a more optimal final solution and simplifies solving optimization problems like TSP by automating it.

3.2.3 Meta-Learning

As mentioned in offline PMO, meta-learning represents an improved technique, and in general, there is a growing trend toward applying machine learning methods. We found that meta learning is used not only in offline PMO but in online PMO too. For example, paper [29] proposed a novel meta-learning-based adaptive operator selection (AOS) framework called LSTM-GA. This approach uses a Long Short-Term Memory (LSTM) neural network to analyze historical performance and dynamic behavioral data from the evolutionary process. By learning temporal patterns, the LSTM model dynamically selects the most effective operators during the run without relying on fixed rules or manual tuning. When tested on the Traveling Salesman Problem (TSP), their method significantly outperformed traditional approaches, achieving a 20% improvement in solution quality (measured by the optimality gap) compared to the best fixed-operator configuration.

Another paper that used online Meta-learning is [51]. This paper introduces BRKGA-QL, a version of the Biased Random-Key Genetic Algorithm (BRKGA) that uses Q-learning (a reinforcement learning technique) to adjust its behavior while running. The Q-agent learns which settings work best at different stages of the search. Crucially, it doesn't only change numerical parameters (like population size or elite ratio) — it also switches between different decoders, which are modules that translate a chromosome into a TSP solution (e.g., using 2-Opt, cheapest insertion, etc.). Because the algorithm can change how solutions are built and interpreted during execution, this represents structure-

level adaptation, not just parameter tuning. Results show that BRKGA-QL found the optimal solution in 72% of TSP instances with an average deviation of 0.14%, while the traditional BRKGA without local search had a much higher deviation of 67.84%, and with local search achieved better results (0.34%). However, in all cases BRKGA-QL produced better solutions than the traditional versions in both speed and performance.

In summary, online PMO (whether deterministic, adaptive, or Meta-learning) provides more flexibility by allowing parameters to dynamically adjust during the run according to the needs of the algorithm at each phase. We can see that all the research discussed in this section concludes that the online PMO strategies they used result in better solutions than the static parameters tuned in similar offline ways, which indicates the effectiveness of online PMO strategies.

3.3 Hybrid PMO

Since we noticed that some papers use both offline and online approaches. Therefore, we named this approach Hybrid Meta-Optimization (HMO). HMO refers to the method that combines the two PMO approaches.

For example in this paper [40] they proposes a two-level (nested) Genetic Algorithm (GA) to solve the Traveling Salesman Problem (TSP) more effectively, the outer GA acts as a meta-optimizer (offline tuning): it searches for the best GA settings like population size, crossover rate, and mutation rate by running the inner GA and using the quality of the solutions it produces as feedback to guide its search, the inner GA is the main solver and uses online adaptive tuning: during the search, it automatically adjusts its crossover and mutation rates based on real-time information, such as how diverse the population is and how quickly the best fitness is improving, by combining offline parameter optimization with online adaptation, the system achieves better solution quality, faster convergence, and more stable performance showing 11–17% improvement over other evolutionary algorithms.

Another relevant study [30] proposes a hybrid metaheuristic framework called VNS.GA.SA.MP.AI. It combines several optimization methods: Variable Neighborhood

Search (VNS), Genetic Algorithm (GA), Simulated Annealing (SA), and POPMUSIC (a partial optimization technique). These methods run iteratively in what serves as the online search process.

The key innovation is the use of an Artificial Neural Network (ANN) to guide the POPMUSIC step. After an initial phase of data collection (called TH), the ANN is trained and then used during the run to decide which parts of the current solution need deeper, mathematically intensive optimization. This makes the ANN a core component of online learning and adaptive decision-making.

Before the main search begins, the algorithm's hyperparameters are set using iRace, an automated configuration tool, this is the offline tuning phase. The results show that VNS.GA.SA.MP.AI significantly outperforms four state-of-the-art methods: it achieves the best and lowest average solution costs across all test instances and even improves on the best-known results for deterministic single-depot CTSP problems. It is also highly efficient ranking as the second-fastest method overall and outperforming three of the four competitors in speed [30]. Across the literature, Table 1 provides a clear summary of the PMO methods reviewed across offline, online, and hybrid studies.

Table 1. Summary Of Used PMO Methods On EAs.

Ref #	Year	PMO approach	Method (sub-approach)	Evalution method	Target Algorithm
[42]	2021	Offline	Meta-EA (Meta-GA)	Random Forest Regression Model (evaluated by Extra Covered Branches + Tuning Gain)	GA
[32]	2025	Offline	Design of Experiments	ANOVA on Gap Metric	GA
[33]	2022	Offline	Design of Experiments	ANOVA on Gap Metric	GA
[34]	2020	Offline	F-Race	Friedman Test on Root Mean Squared Error (RMSE)	GP

[47]	2017	Offline	HORA	Racing Evaluation on Gap Metric	GA and SA
[45]	2024	Offline	Surrogate Model (Random Direction Hill-Climbing - RHC)	Loss Metric	GA
[44]	2020	Offline	Surrogate Model (Bayesian Optimization)	Acquisition Function evaluated by Relative Error	GGA
[28]	2016	Offline	Meta learn (Label Ranking)	Friedman Test on Spearman Correlation	Meta-heuristics
[41]	2017	Offline	Meta learn (Genetic Programming Hyper-heuristic (GPHH))	Mann-Whitney-Wilcoxon Test on Distance Metric	GA
[46]	2023	Offline	Meta-Black-Box Optimization (OpenAI-ES)	Aggregated Meta-Fitness (Z-Score + Median)	GA → LGA
[4]	2022	Online	Deterministic control for three parameters and Adaptive control for GA	Pearson Correlation with Fitness (Shortest Distance) and Execution Time	Adaptive GA
[48]	2019	online	Deterministic (ILM/DHC) - (DHM/ILC)	Fitness (Min Distance)	GA
[50]	2020	online	Adaptive (PRoFIFEA (Fuzzy Logic + PRoFIGA concept))	Experimental Evaluation on Fitness Value (derived from Route Length)	GA / PRoFIFEA
[29]	2025	Online	Meta learn (LSTM Neural Network)	Optimality Gap (Minimized)	GA / LSTM-GA
[51]	2021	Online	Meta-Learning (Q-Learning (QL) / RVND (Local Search))	Relative Percentage Deviation (RPD) and Optimality Gap	BRKGA / BRKGA-QL
[30]	2026	Hybrid	VNS.GA.SA.MP.AI (POPMUSIC guided by ANN and tuned by iRace)	Robust Objective Function (Minimizing sum of squared deviations from $f*$)	Matheuristic (VNS, GA, SA, POPMUSIC)
[40]	2024	Hybrid	Meta-GA (Outer layer), Adaptive Parameter Adjustment Mechanisms (Inner layer)	Fitness (Minimum, Average), Optimal Solution Quality (%)	GA

3.4 Summary

Parameter Meta-Optimization (PMO) techniques fall into three categories: offline, online, and hybrid. Offline methods such as Design of Experiments, surrogate-based models, and meta-learning tune parameters before execution and provide informed initial settings, but they are slow and computationally expensive. Online methods adjust parameters dynamically during execution, either through fixed rules (deterministic control) or performance feedback (adaptive control), offering greater flexibility and better performance, especially on problems like the TSP. Recent studies show a clear shift toward online adaptation, with meta-learning increasingly used to guide dynamic tuning. However, the literature indicates that even advanced online approaches often start with default or uninformed initial parameters, which wastes early search effort and delays convergence. Hybrid methods address this limitation by combining offline intelligence with online responsiveness, achieving a balance between informed initialization and dynamic adjustment.

Building on this trend, our work proposes a novel Hybrid Meta-Optimization framework that integrates meta-learning such as Deep Neural Networks for offline parameter prediction and an adaptive Genetic Algorithm for online tuning during the search process. This approach is promising and innovative because it combines the strength of offline intelligence, which enables a well-informed initial setup, with online responsiveness, which allows dynamic adaptation during the search, to enhance GA performance on the TSP. The next chapter presents the methodology of the proposed framework.

Chapter 4: Methodology

This chapter presents the methodology used to design and implement the Hybrid Parameter Meta-Optimization framework for solving the Traveling Salesman Problem (TSP). The approach is based on combining both offline and online parameter optimization techniques to enhance the performance of the Genetic Algorithm (GA). The offline component uses a Neural Network (NN) to predict suitable initial GA parameters based on features collected from the TSP instance, while the online component applies an Adaptive Genetic Algorithm (AGA) that adjusts these parameters dynamically during the search.

The chapter is organized into several parts. First, the representation of the TSP and the evaluation measures used throughout the study are defined. Then, the overall workflow of the hybrid framework is presented, showing how the offline and online components interact. The offline phase explains how the NN is designed, trained, and used to predict GA parameters based on features collected from TSP instances. Following that, the online phase describes how the AGA adjusts key parameters, population size, crossover rate, and mutation rate during the run time. Finally, the chapter outlines the solution representation used within the GA and provides a summary.

4.1 Problem Representation

In this study, the Traveling Salesman Problem (TSP) is represented as a set of two-dimensional coordinates, where each city i is defined by its position:

$$City_i = (x_i, y_i)$$

Thus, the full TSP instance is expressed as:

$$TSP\ Instance = \{City_1, City_2, \ldots, City_n\} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

The objective is to find the shortest possible tour that visits every city exactly once and returns to the starting point. The distance D between any two cities i and j is computed using the Euclidean formula:

$$D(i, j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$

Because the number of possible tours grows factorially with the number of cities, exact algorithms become impractical for larger instances. For this reason, meta-heuristic algorithms, particularly Genetic Algorithms (GAs), are widely used to obtain high-quality solutions efficiently. A candidate solution found by the GA is represented as a route:

$$R = [r_1, r_2, \ldots, r_n]$$

where each r_i denotes the index of the city visited at position i in the tour and n is the total number of cities.

The quality of a route is evaluated using the fitness function, which in the case of the TSP corresponds to the total tour length. To measure how close the GA solution is to the optimal or best-known value:

Fitness Function:

$$f(R) = \sum_{i=1}^{n-1}\left(D(r_i, r_i{}^{+1})\right) + D(r_n, r^1)$$

Gap Function:

$$GAP\ (\%) = \frac{\text{Obtained Tour Length} - \text{Known Optimal Tour Length}}{\text{Known Optimal Tour Length}} \times 100$$

GA relies on key parameters such as population size, crossover rate, and mutation rate, and their performance is strongly influenced by how these parameters are set. Many methods have been developed for parameter optimization, including offline approaches based on experimental analysis or machine learning models, and online approaches that adjust parameters adaptively during the run. Although both categories have been widely explored in the literature, only a few studies have combined them into a single hybrid method.

This study addresses this gap by introducing a Hybrid Parameter Meta-Optimization framework. The approach uses a NN to predict suitable initial parameters offline and an AGA to adjust these parameters online during the search. TSP features are first extracted for the NN to estimate starting values, after which the AGA refines these values based on performance feedback. The goal is to improve solution quality,

convergence behavior, and overall optimization efficiency while assessing the effectiveness of integrating an NN with an adaptive GA.

4.2 Framework Overview

The proposed framework aims to improve the performance of GA in solving the TSP by combining both offline and online Parameter Meta-Optimization techniques. The framework consists of two main components: an Offline Neural Network Meta-Optimizer and an Online Adaptive Genetic Algorithm. These two components work together to provide effective parameter settings throughout the optimization process.

In the offline phase, the TSP instance is first represented as a set of two-dimensional coordinates *(x, y)* for each city. From this representation, a set of descriptive features is extracted, such as the number of cities or average inter-city distances. These features are then used as inputs to a trained Neural Network model, which predicts suitable initial values for key GA parameters, including population size, crossover rate, and mutation rate. The purpose of this phase is to allow the algorithm to start from informed parameter values rather than relying on manual trial-and-error or arbitrary defaults.

In the online phase, the optimization process is carried out using an Adaptive Genetic Algorithm. While the GA searches for a near-optimal tour through selection, crossover, mutation, and elitism, its parameters are adjusted dynamically based on performance feedback. For example, mutation rate may increase when progress slows to help escape local optima, while population size may decrease over time to reduce computation. This adaptive behavior enables the algorithm to respond to changes in search conditions and improve convergence stability.

The integration of these two components forms a hybrid Meta-Optimization framework as shown in Fig. 8. The offline NN provides a strong initialization, and the online Adaptive GA continually refines parameter values during the search. Together, they aim to achieve better solution quality, faster convergence, and greater robustness across different TSP instances compared with using either method alone.

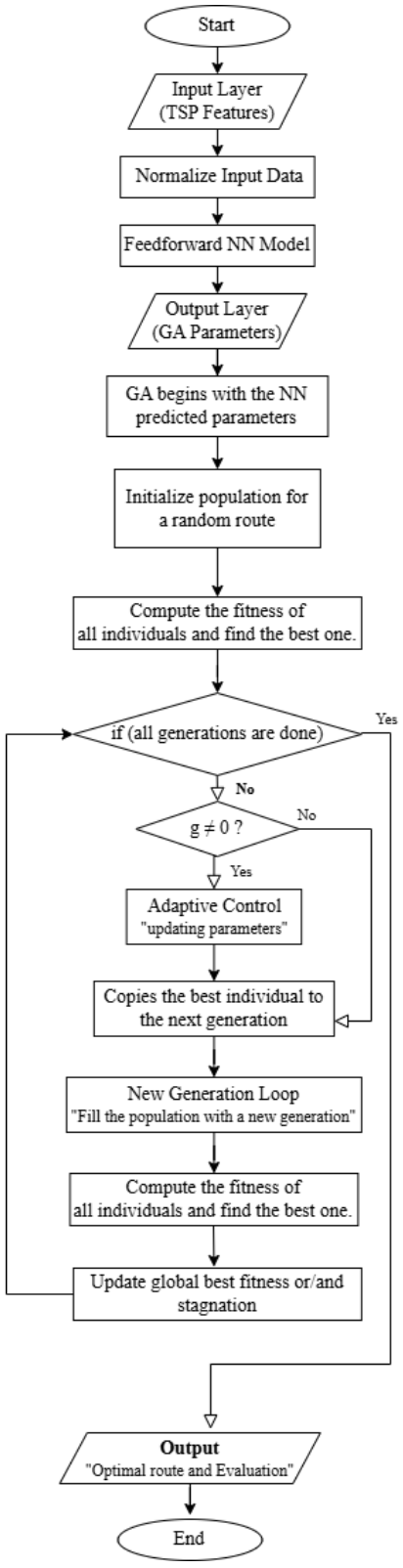

Fig. 8. Hybrid Meta-Optimization for GA parameters for solving TSP.

4.3 Neural Network Offline phase

The offline component of the Hybrid Meta-Optimization framework uses a Neural Network (NN) to predict suitable initial parameter values for the Genetic Algorithm before execution. The main goal of this module is to provide the GA with an informed starting point, instead of relying on manual tuning or default settings. Although Neural Networks have been widely used in online control, in this study we will use it as an offline meta-optimizer for GA parameters. Therefore, this study explores the use of an NN for static (offline) prediction of GA parameters based on features extracted from TSP instances.

The NN acts as a regression model that maps characteristics of a given TSP instance, such as the number of cities or statistical properties of the distance matrix, to recommend values for population size, crossover rate, and mutation rate. These predictions serve as the initial settings for the Adaptive GA, allowing it to begin the search for more suitable parameter values.

4.3.1 Neural Network Architecture:

The Neural Network is designed as a lightweight feedforward model suitable for small- to medium-scale datasets. The structure contains three main components:

- **Input Layer:**

The input layer receives descriptive features of the TSP instance. These features were chosen because they affect how the GA behaves when searching for good solutions. The input layer contains the following features:

1. Number of cities.
2. TSP type (e.g., symmetric or asymmetric, encoded numerically).
3. Mean inter-city distance

Before training, all input features are normalized using Min–Max scaling to the range [0,1] to ensure consistent magnitude and improve network convergence

- **Hidden Layers:**

In this study, the network initially employs a single hidden layer composed of eight neurons. This compact configuration serves as a primary baseline, allowing us to evaluate its ability to capture the relationship between TSP instance features and optimal GA parameters. Depending on the observed performance, the architecture can be further refined or expanded to improve predictive accuracy.

- **Output Layer:**

The output layer contains three neurons corresponding to the GA parameters to be predicted:

1. Population Size (PS)
2. Mutation Rate (MR)
3. Crossover Rate (CR)

A linear activation function is used in the output layer since the model performs regression. Its mathematical definition is:

$$(2) f(x) = x$$

This architecture was selected because it offers a balance of simplicity, interpretability, and generality. Its simple structure helps prevent overfitting and reduces training time, making it efficient to train even with limited data. At the same time, the model remains interpretable, allowing the relationship between problem features and the predicted parameters to be easily understood. Finally, the architecture maintains good generality, enabling it to generalize across different TSP instances without requiring unnecessary complexity.

4.3.2 Neural Network Training and Testing:

The NN will train offline prior to running the GA. The training methodology consists of the following steps:

1.Dataset Preparation:

A dataset will be constructed using previously solved TSP instances. The dataset is expected to contain approximately 200–500 instances, depending on availability from TSPLIB and generated synthetic instances.

For each instance, the following will be recorded:

- TSP features (inputs) such as number of cities, TSP type, and mean inter-city distance.
- Best-performing GA parameters (outputs).

Since GA parameter values are not provided by the dataset, they will be obtained experimentally by running the GA with multiple parameter combinations and selecting the configuration that produces the shortest tour length, which corresponds to the minimum value of the TSP evaluation function. This parameter set is treated as the "best-performing" output for that instance.

2.Data Splitting:

The dataset will be divided as follows: 70% for training, 15% for validation, and 15% for testing, ensuring that the model generalizes well to unseen TSP instances.

3.Training Procedure:

The neural network will be implemented and trained using the TensorFlow/Keras deep-learning library. Training will be performed using the Adam optimizer with a learning rate of 0.001, over 50–100 epochs, and with a batch size of 16–32, depending on validation performance. These hyper-parameters are chosen to ensure stable convergence and efficient learning.

The model will be trained using the Mean Squared Error (MSE) loss function, which measures the average squared difference between the predicted parameter values and the true (best-performing) values. MSE is appropriate for regression problems

because it penalizes larger errors more heavily and encourages stable convergence of the network.

$$MSE = \left(\frac{1}{n}\right)\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- n = number of samples
- y_i = true target value
- $\hat{y}_i$ = predicted value from the neural network

4.Model performance:

will be evaluated using validation MSE and test MSE. These values ensured that the NN could generalize well to unseen TSP instances.

5.Final Output:

Once training is complete, the NN produces predicted GA parameter values for any new TSP instance. These parameters are continuous floating-point values within predefined ranges: population size is predicted in the range [50, 300], crossover rate in [0.5, 1.0], and mutation rate in [0.01, 0.2]. Although population size is an integer in practice, the NN outputs a floating-point approximation that is rounded to the nearest valid integer before being used. These predicted values are then used as the initial parameters for the Adaptive GA in the hybrid method.

4.4 Online Adaptive phase

After the offline phase provides the initial values of the selected parameters to the AGA, the online phase begins, where the AGA is applied to solve the main optimization problem, which is the TSP instance itself.

The overall structure of the AGA follows the standard GA framework, but the main difference is its ability to adapt its parameters dynamically during the search. This adaptive behavior allows the algorithm to respond to the different needs of the search process at different stages and helps avoid premature convergence and maintain diversity

In the proposed framework, the AGA will control population size (PS), crossover rate (CR) and mutation rate (MR). Building on the previous explanation, the following methodology outlines the main stages of the algorithm and the adaptive logic of the algorithm.

First the AGA will receive as input the TSP instance and the initial values of PS, CR, MR provided by the previous online phase, then:

1. Initialization

   The algorithm generates an initial population of random tours based on the initial value of PS. All individuals are evaluated then the best individual is determined and the best fitness value is stored.

2. Adaptive Parameter Control

   The three key parameters are adapted in all generations, except the first, which uses values from the offline phase. This adaptation balances exploration (searching new areas of the solution space) and exploitation (enhancing the current area of the solution space) as follows:

1. High CR at the start: to exploit the initial solutions, as the first population is randomly initialized with high diversity so the resulting variation may appear as exploration

2. Stagnation: if a certain percentage of generations show no improvement, PS and CR are increased to inject diversity.

3. Continued stagnation: MR is increased to escape local optima.

4. After overcoming stagnation: CR decreases over time to stabilize solutions.

5. PS: generally, decreases over time to reduce computational cost and focus on the best individuals, except when diversity is needed.

The adaptation of each parameter is explained in detail below:

- **Crossover Rate (CR):**

$$CR = CR_{max} - (CR_{max} - CR_{min}) * (g/G) \quad (4)$$

if stagnation exceeds 7% of the total generations, CR is temporarily increased:

$$CR = \min(CR_{max}, CR + 0.1) \quad (5)$$

Where CR_{max} and CR_{min} predefined upper and lower bounds of the CR. Stagnation, as mentioned, is the number of generations without any improvement in fitness. g is the current generation, and G is the total number of generations. The CR starts high to exploit the primitive population, which often has high diversity, producing new solutions that may appear as exploration. It then decreases linearly over time to focus on stabilizing solutions and maintaining the structure of existing good solutions. According to (5), if stagnation occurs, the CR will increase to exploit the new diversity in the population. Once the stagnation is overcome, the CR will start to decrease again over time.

- **Population Size (PS):**

$$PS = PS_{max} - (PS_{max} - PS_{min}) * (g/G) \quad (6)$$

if stagnation exceeds 7% of the total generations, PS is temporarily increased:

$$PS = \min\left(PS_{max},\ PS + int(0.1 * PS_{max})\right) \quad (7)$$

According to (7), when stagnation occurs, diversity is injected by increasing the PS. According to (6), PS decreases linearly over time to reduce computational cost and focus on the best individuals, except when diversity needs to be injected.

- **Mutation Rate (MR):**

$$MR = MR_{min} + (MR_{max} - MR_{min}) * \min\left(1,\ stagnation/(0.1 * G)\right) \quad (8)$$

MR increases as stagnation grows. It reaches its maximum value when 10% of the total generations show no improvement in fitness. MR thus tries to overcome the local optimum, especially when stagnation continues, despite previous attempts to overcome it by increasing PS and CR.

These adaptive components enable the GA to dynamically adjust its search behavior based on real-time performance.

3. Population Update (Generation of New Population)

The next generation is constructed as follows:

- Elitism: The best individual from the previous generation is copied directly into the new generation.

Remaining individuals are generated through:

- Tournament selection: For each generated individual, tournament selection is applied twice, once to select each parent.
- Partially Mapped Crossover (PMX) (conditional on CR)
- Inversion mutation (conditional on MR)

These operators were chosen because they are commonly applied and effective for permutation-based problems such as the TSP. Tournament Selection combines a higher chance for stronger individuals while maintaining diversity by giving weaker individuals a chance; it also allows balancing between exploration and exploitation by controlling the tournament size. PMX was chosen because it preserves city order and prevents duplication, which generates valid routes for TSP. Inversion mutation is widely used in TSP because reversing part of a route can lead to shorter distances. This combination of operators tries to balance effectiveness and logical run time.

This process continues until the new population reaches the adaptive PS.

4. Best Solution Update and Stagnation Handling

After generating the new population, the current best individual is identified. It is then compared with the global best to either update the global best or increase the stagnation counter.

5. Termination Criterion

The algorithm iterates for a fixed number of generations. Once the maximum generation count is reached, the search terminates.

After these stages, the output of the algorithm will be the best tour found during the entire process is reported, along with its corresponding fitness value (distance). A full outline of these steps is presented in Algorithm 1.

Algorithm 1 *ADAPTIVE_GENETIC _ALGORITHM (AGA)*

```
Input: Initial CR, MR, PS from offline phase; TSP instance
Output: Best-found tour and its fitness
population ← InitializePopulation(PS)
best ← best individual in population
best_fit ← fitness(best)
stagnation ← 0
For g = 0 to GENERATIONS-1 do
    If g ≠ 0 then
        MR, CR ← adaptive_MR_CR(g, best_fit, stagnation)
        PS ← adaptive_population_size(g, stagnation)
    new_pop ← [best]   # elitism
    While size of new_pop < PS do
        p1 ← TournamentSelection(population)
        p2 ← TournamentSelection(population)
        If random() < CR then
            child ← PMX_Crossover(p1, p2)
        Else
            child ← copy of p1
        If random() < MR then
            Apply InversionMutation(child)
        Add child to new_pop
    population ← new_pop
    current_best ← best individual in population
    current_fit ← fitness(current_best)
    If current_fit > best_fit then
        best ← current_best
        best_fit ← current_fit
        stagnation ← 0
    Else
        stagnation ← stagnation + 1
Return best solution and best_fit
```

In conclusion, the AGA will be applied after the online phase to solve the TSP instance. It takes the TSP instance and the initial parameters as input and gives the best tour with its fitness as output. Like a normal GA, it starts by creating a random population, finds the best fitness, updates the population using elitism, tournament selection, PMX crossover, and inversion mutation, and chooses the best local and global solutions. The difference is that the AGA adapts its parameters (PS, CR, MR) for every generation except the first. The AGA starts with a high CR and decreases over time. If

stagnation is detected, PS and CR are increased to add diversity, and if stagnation continues, MR reaches its maximum value to escape local optima. Once stagnation is overcome, the CR starts to decrease again. This method allows the AGA to solve the TSP more efficiently by starting with good initial values and adjusting the parameters during the search to match the needs of each stage.

4.5 Summary

This chapter presented the complete methodology used to develop the Hybrid Parameter Meta-Optimization framework for solving the TSP. The chapter introduced the overall structure of the system, which integrates an offline Neural Network meta-optimizer with an online Adaptive Genetic Algorithm. The TSP instance is first represented using two-dimensional coordinates, and useful features are extracted for the NN crossover rate, and mutation rate. These predicted values are then used to initialize the GA. During execution, the Adaptive GA adjusts its parameters continuously based on stagnation and search performance using adaptive rules. The chapter also described the GA operators used in the system, including tournament selection, PMX crossover, and inversion mutation. Together, these components form a hybrid framework that supports both informed initialization and dynamic, feedback-based adjustment. Overall, Chapter 4 establishes the methodological foundation for building and evaluating the proposed optimization system.

Chapter 5: Experimental Design

5.1 Dataset:

To evaluate the proposed hybrid meta-optimization framework, two datasets are used: one for solving the TSP instances using the GA and one for training the NN.

5.1.1 TSPLIB Benchmark Instances

TSPLIB is a widely used collection of benchmark instances as shown in *Table 2.* for the TSP and related combinatorial optimization problems. It is considered the standard testing ground for TSP algorithms in academic and research communities [53].

Each instance in TSPLIB is provided as a structured text file that includes:

- Problem metadata: such as the instance name, type (e.g., symmetric, asymmetric), and number of cities (dimension).
- City representation: 2D coordinates (x, y) for each city.
- Distance specification: Euclidean distances computed from the 2D coordinates.

The data includes:

- Unique node IDs for each city,
- Geographical coordinates (x, y),
- Precomputed Euclidean distances between cities, derived from the coordinates.

For this study, we focus on small-to-medium-scale TSP instances. This range ensures computational feasibility while still providing diverse and representative problem structures for evaluating algorithm performance, as is standard in related work.

Table 2. Experimental Dataset Table (TSPLIB Instances)

Instance Name	# Cities	TSP Type	(x, y) coordinates
berlin52	52	Symmetric	[(1380, 939), (2848, 96), ...]
eil51	51	Symmetric	[(37, 52), (49, 49), (52,64), ...]
eil76	76	Symmetric	[(22, 22), (36, 26), (21,45), ...]

5.1.2 Neural Network Training Dataset

Since there is no ready dataset that fits our NN, the training dataset is collected from multiple solved TSP instances. Each record in the dataset contains input features that describe the TSP instance, along with the best GA parameters found for that instance.

The dataset includes the following features:

Inputs (TSP instance features):

- Number of cities
- TSP Type
- Mean inter-city distance

Outputs (optimal GA parameter settings):

- Population Size
- Crossover Rate
- Mutation Rate

This dataset is used just for training and evaluating the NN, so it can correctly predict suitable initial GA parameters for new TSP instances.

Table 3. Experimental Dataset Table (NN Training Dataset)

# Cities	TSP Type	Mean Distance	PS	CR	MR
50	Symmetric	120.4	150	0.85	0.10
75	Symmetric	135.2	180	0.80	0.15
100	Symmetric	140.8	200	0.75	0.20

5.2 Parameters

In this, we focus on three key control parameters of the Genetic Algorithm (GA), as they directly affect how the algorithm searches for good solutions to the Traveling Salesman Problem (TSP):

- Crossover Rate (CR):

The probability that two selected tours exchange parts of their routes to produce a new tour.

- Mutation Rate (MR):

The probability that a small random change is applied to a tour after it is created.

- Population Size (PS):

The number of different tours maintained in each generation.

These parameters will be tuned using a hybrid approach:

- Offline Tuning:

Before the GA starts, a pre-trained neural network predicts initial values for CR, MR, and PS. The network takes input features of the TSP instance, such as the number of cities and the symmetry type—and outputs recommended parameter values. These values are not fixed for all problems; instead, they are learned from experience and tailored to the specific characteristics of each TSP instance.

- Online Tuning:

During execution, an adaptive GA adjusts the parameters dynamically based on real-time feedback from the search process, allowing the algorithm to modify its behavior in response to the current state of the search. This adaptation is constrained within fixed minimum and maximum boundaries for each parameter. These boundaries are used in the

adaptation equations, help maintain search stability, and prevent parameters from becoming too high or too low. To ensure a systematic and fair comparison across all TSP instances, the same boundaries are applied universally in this study. They are determined based on commonly used default values in the GA literature and are consistent with the problem size (i.e., number of cities) considered in our experiments.

This hybrid strategy aims to start with smart initial settings and remain flexible enough to respond to each TSP instance's unique characteristics.

5.3 Evaluation Metrics

The performance of our framework will be assessed using the following metrics:

1. **Tour Length (Fitness Function)**

The total distance of the best tour found. This is the main objective of the optimization: the shorter the tour, the better the solution.

2. **Execution Time:**

The wall-clock time (in seconds) required to reach the final solution. This reflects computational efficiency.

3. **Algorithmic Complexity:**

The theoretical time complexity of the proposed approach is O (G × P × n) for the online Genetic Algorithm, where G is the number of generations, P is the population size, and n is the number of cities. The offline meta-learning step adds a negligible fixed cost, as it is computed once per instance.

4. **Solution Quality:**

Measured using the optimality gap, calculated as:

$$\text{Optimality Gap (\%)} = \frac{\text{Obtained Tour Length} - \text{Kown Optimal Tour Length}}{\text{Known Optimal Tour Length}} \times 100$$

A smaller gap indicates higher solution quality.

5.4 Hypothesis

Our research addresses the following question:

Does adding an offline meta-learning step before running the adaptive Genetic Algorithm (GA) significantly improve performance compared to using online adaptation alone?

To evaluate this, we compare two configurations of the same adaptive GA:

- Online-only:
  The GA starts with fixed initial parameter values and relies only on real-time feedback during execution to adjust its parameters.

- Hybrid:
  The GA starts with initial parameter values predicted by a pre-trained neural network, based on features of the TSP instance (such as number of cities, symmetry type, and distance statistics). It then applies the same online adaptation rules as in the online-only mode during the search.

Both configurations share the same GA structure, operators, adaptive rules, and stopping conditions. The only difference is that the hybrid mode starts with meta-learning–predicted parameters, whereas the online-only mode starts with fixed defaults.

If the hybrid mode consistently outperforms the online-only mode across multiple TSP instances, measured by tour length, execution time, or solution quality, we will consider the hypothesis supported.

5.5 Summary

Chapter 5 outlines the experimental design for evaluating the proposed hybrid meta-optimization framework. The study compares two configurations of the same Adaptive Genetic Algorithm: one initialized with fixed default parameters (online-only), and another initialized with parameters predicted by a pre-trained neural network based on TSP instance features such as number of cities, problem type, and mean inter-city distance (hybrid). Both configurations use identical operators, adaptive rules, and termination criteria. Performance will be measured using tour length, execution time, and optimality gap on standard TSPLIB instances (e.g., berlin52, eil51, eil76). The neural network will be trained on a custom dataset linking TSP features to high-performing GA parameter settings. Since initialization is the only difference, any consistent improvement in the hybrid can be attributed to meta-learning–guided initialization, directly testing the core hypothesis.

Chapter 6: Conclusion

This study presented a Hybrid Parameter Meta-Optimization framework for improving the performance of GA in solving the TSP. The main goal was to address the sensitivity of GA performance to parameter settings such as population size, crossover rate, and mutation rate. Instead of relying on manual tuning, the proposed method combines both offline and online optimization techniques to provide more reliable parameter values.

In the offline phase, a NN model will be used to predict initial GA parameters based on features extracted from each TSP instance. These learned predictions offer an informed starting point and reduce the need for repeated experimentation. In the online phase, an Adaptive Genetic Algorithm continuously adjusts its parameters according to the algorithm's progress. This dynamic control helps maintain diversity, overcome stagnation, and guide the search more effectively toward high-quality tours.

Using benchmark datasets from TSPLIB allowed the framework to be evaluated on well-known and diverse TSP instances. The methodology demonstrated how offline meta-learning and online adaptive strategies can complement each other: offline prediction provides stability and good initialization, while online adaptation enables the algorithm to respond to real-time search behavior. Together, these two components work in a coordinated manner to enhance convergence efficiency and improve the quality of the final solutions.

Overall, this study represents the first phase of our research, where we reviewed existing methods and identified a promising direction for future implementation. The literature shows that both meta-learning approaches and adaptive control techniques have produced strong results when used separately for parameter optimization. Based on these findings, we propose combining them into a hybrid meta-optimization framework. The next phase of this research will focus on implementing and experimentally evaluating the proposed hybrid approach. In addition, we will test different structures of the NN model, including the one suggested in the methodology. In conclusion, this project provides a conceptual foundation for an automated parameter-tuning method for Genetic Algorithms and related optimization techniques.

References

[1] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, *3rd ed*. Boston, MA, USA: Pearson Addison-Wesley, 2012.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. St*ein, Introduction to Algorithms, 3rd ed. Cambridge*, MA, USA: MIT Press, 2022.

[3] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA, USA: MIT Press, 1996.

[4] I. K. Herdiana, I. M. Candiasa, and G. Indrawan, "Optimization of Adaptive Genetic Algorithm Parameters in Traveling Salesman Problem," *CNAHPC*, vol. 4, no. 2, pp. 169–176, Jul. 2022.

[5] S. K. Smit and A. E. Eiben, ".," in Proc. IEEE Congress on Evolutionary Computation (CEC), Trondheim, Norway, May 2009, pp. 399–406.

[6] S. Mirjalili, J. S. Dong, A. S. Sadiq, and H. Faris, "A Bayesian Optimization Approach for Tuning a Genetic Algorithm Solving Practical-Oriented Pickup and Delivery Problems," *Logistics*, vol. 8, no. 1, p. 14, Jan. 2024.

[7] L. Calvet, A. de Armas, D. Masip, and A. Juan, "A survey of automatic parameter tuning methods for metaheuristics," *Int. Trans. Oper. Res*., vol. 27, no. 2, pp. 117–143, Mar. 2020.

[8] C. Neumüller, S. Wagner, G. Kronberger, and M. Affenzeller, "Parameter meta-optimization of metaheuristic optimization algorithms," in *Computer Aided Systems Theory – EUROCAST 2011*, R. Moreno-Díaz, F. Pichler, and A. Quesada-Arencibia, Eds., *Lecture Notes in Computer Science*, vol. 6927, Springer, 2012, pp. 367–374.

[9] C. Huang, Y. Li, and X. Yao, "A Survey of Automatic Parameter Tuning Methods for Metaheuristics," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 2, pp. 201–216, Apr. 2020.

[10] P. C. Pop, O. Cosma, C. Sabo, and C. P. Sitar, "A comprehensive survey on the generalized traveling salesman problem," *European Journal of Operational Research*, vol. 314, no. 3, pp. 819–835, 2024.

[11] M. Jünger, G. Reinelt, and G. Rinaldi, "The traveling salesman problem," *Handbooks in Operations Research and Management Science*, vol. 7, pp. 225–330, 1995.

[12] S. Saller, J. Koehler, and A. Karrenbauer, "A survey on approximability of traveling salesman problems using the TSP-T3CO definition scheme," *Annals of Operations Research*, pp. 1–62, 2025.

[13] R. Sorma, M. A. Wadud, S. R. Karim, and F. S. Ahamed, "Solving traveling salesman problem by using genetic algorithm," *Electrical and Electronic Engineering,* vol. 10, no. 2, pp. 27–31, 2020.

[14] P. Das and A. Banerjee, “Meta optimization and its application to portfolio selection,” in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’11)*, San Diego, CA, USA, Aug. 21–24, 2011, pp. 1163–1171.

[15] S. N. Sivanandam and S. N. Deepa, *Introduction to Genetic Algorithms*. Berlin, Germany: Springer, 2007.

[16] S. Boyd and L. Vandenberghe*, Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[17] F. S. Hillier and G. J. Lieberman*, Introduction to Operations Research*, 9th ed. New York, NY, USA: McGraw-Hill, 2010.

[18] E.-G. Talbi, Metaheuristics*: From Design to Implementation*. Hoboken, NJ, USA: Wiley, 2009.

[19] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.

[20] N. Christoforou, “Chapter 10: The Traveling Salesman Problem,” *HY-583 Algorithms Lecture Notes*, Department of Computer Science, University of Crete, Heraklion, Greece.

[21] T. Alam, S. Qamar, A. Dixit, and M. Benaida, “Genetic Algorithm: Reviews, Implementations, and Applications,” *Int. J. Eng. Pedag.*

[22] L. Gao and S. Liu, "Comparing parameter tuning methods for evolutionary algorithms*," Comput. Oper. Res.*, vol. 135, p. 105466, 2021.

[23] M. Mosayebi and M. Sodhi, “Tuning genetic algorithm parameters using design of experiments,” in *Proc. 2020 Genetic and Evolutionary Computation Conference Companion (GECCO ’20 Companion)*, Cancún, Mexico, Jul. 2020.

[24] F. Kharroubi, “*Random Search Algorithms for Solving the Routing and Wavelength Assignment in WDM Networks*,” Ph.D. dissertation, Hunan Univ., Changsha, China, 2014.

[25] S. de León-Aldaco, H. Calleja, and J. Aguayo, “Metaheuristic Optimization Methods Applied to Power Converters: A Review,” *IEEE Transactions on Power Electronics*, vol. 30, no. 12, pp. 6791–6803, Jan. 2015.

[26] J. Branke and J. A. Elomari, “Meta-optimization for parameter tuning with a flexible computing budget,” in *Proceedings of the 14th International Conference on Genetic and Evolutionary Computation (GECCO ’12)*, Philadelphia, PA, USA, July 7–11 2012, pp. 1245-1252.

[27] B. S. Petersen and J. M. Marsden, *Introduction to Optimization*. New York, NY, USA: Springer-Verlag, 2004.

[28] J. Kanda, A. de Carvalho, E. Hruschka, C. Soares, and P. Brazdil, "Meta-learning to select the best meta-heuristic for the Traveling Salesman Problem: A comparison of meta-features," *Neurocomputing*, vol. 205, pp. 393–406, 2016.

[29] H. Y. Jeong and B. D. Song, "Meta-learning-based adaptive operator selection for traveling salesman problem," *Applied Soft Computing*, vol. 185, Article 113930, 2025.

[30] A. Nourmohammadzadeh and S. Voß, "A matheuristic approach for the robust coloured travelling salesman problem with multiple depots," *European Journal of Operational Research*, vol. 328, pp. 390–406, 2026.

[31] M. Mosayebi and M. Sodhi, "Tuning genetic algorithm parameters using design of experiments," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion (GECCO '20 Companion)*, Cancún, Mexico, Jul. 2020, pp. 1937-1944.

[32] W. El Mestari, N. Cheggaga, F. Adli, A. Benallal, and A. Ilinca, "Tuning parameters of genetic algorithms for wind farm optimization using the design of experiments method," *Sustainability*, vol. 17, no. 7, art. no. 3011, Mar. 2025.

[33] N. Nisrina, M. I. Kemal, I. A. Akbar, and T. Widianti, "The Effect of Genetic Algorithm Parameters Tuning for Route Optimization in Travelling Salesman Problem through General Full Factorial Design Analysis," *Evergreen Joint Journal of Novel Carbon Resource Sciences & Green Asia Strategy*, vol. 9, no. 1, pp. 163-203, Mar. 2022.

[34] L. Trujillo, E. Álvarez González, E. Galván, J. J. Tapia, and A. Ponsich, "On the analysis of hyper-parameter space for a genetic programming system with iterated F-Race," *Soft Computing*, vol. 24, no. 19, pp. 14757-14770, 2020

[35] A. P. Punnen, "The traveling salesman problem: Applications, formulations and variations," in The Traveling Salesman Problem and Its Variations, Boston, MA: Springer US, 2007, pp. 1–28.

[36] H. Jiang, "The 1-1 algorithm for Travelling Salesman Problem," *arXiv preprint arXiv:2104.13197*, 2021.

[37] G. Laporte, "The traveling salesman problem: An overview of exact and approximate algorithms," *European Journal of Operational Research*, vol. 59, no. 2, pp. 231–247, 1992.

[38] D. Davendra, Ed., Traveling Salesman Problem: Theory and Applications, *InTech*, 2010.

[39] O. Kramer, *Genetic Algorithm Essentials*. Cham, Switzerland: Springer International Publishing, 2017, pp. 11–19.

[40] Q. Chen and G. Gong, “Adaptive parameter tuning with double layer genetic algorithm for solving TSP,” in *Proc. 25th Int. Arab Conf. Information Technology (ACIT)*, Dec. 2024, pp. 1–7.

[41] E. Sopov, C. Sabourin, J. J. M. Guervos, U. M. O’Reilly, K. Madani, and K. Warwick, “Genetic programming hyper-heuristic for the automated synthesis of selection operators in genetic algorithms,” in *Proc. Int. Joint Conf. Computational Intelligence (IJCCI)*, Nov. 2017, pp. 231–238.

[42] S. Zamani and H. Hemmati, “A pragmatic approach for hyper-parameter tuning in search-based test case generation,” *Empirical Software Engineering*, vol. 26, no. 6, p. 126, 2021.

[43] A. A. Chaves and L. H. N. Lorena, "An adaptive and near parameter-free BRKGA using Q-learning method," in *2021 IEEE Congress on Evolutionary Computation (CEC)*, Kraków, Poland, 2021, pp. 1-8.

[44] C. Rüther and J. Rieck, "A Bayesian optimization approach for tuning a genetic algorithm solving practical-oriented pickup and delivery problems,"*Oper. Res. Perspect*., vol. 9, p. 100821, 2022.

[45] C. Kim and I. Joe, “A balanced approach of rapid genetic exploration and surrogate exploitation for hyperparameter optimization,” *IEEE Access*, vol. 12, pp. 192184-192194, 2024.

[46] R. T. Lange, T. Schaul, Y. Chen, C. Lu, T. Zahavy, V. Dalibard, and S. Flennerhag, “Discovering attention-based genetic algorithms via meta-black-box optimization,” in *Proceedings of the 2023 Genetic and Evolutionary Computation Conference (GECCO ’23)*, Lisbon, Portugal, July 15–19 2023, ACM, 14 pages

[47] E. B. de Moraes Barbosa and E. L. F. Senne, “Improving the fine-tuning of metaheuristics: An approach combining design of experiments and racing algorithms,” *Journal of Optimization*, vol. 2017, pp. 1-7, June 2017.

[48] A. Hassanat, K. Almohammadi, E. Alkafaween, E. Abunawas, A. Hammouri, and V. B. S. Prasath, "Choosing Mutation and Crossover Ratios for Genetic Algorithms—A Review with a New Dynamic Approach," *Information*, vol. 10, no. 12, p. 390, 2019, doi: 10.3390/info10120390.

[49] M. Ryngier and U. Boryczka, "Tuning or Control Parameter Values in Evolutionary Art," *Procedia Computer Scienc*e, vol. 246, pp. 5064–5073, 2024.

[50] S. Muzid, "An Adaptive Approach to Controlling Parameters and Population Size of Evolutionary Algorithm," *Journal of Physics: Conference Series*, vol. 1430, p. 012048, 2020.

[51] A. A. Chaves and L. H. N. Lorena, "An Adaptive and Near Parameter-Free BRKGA Using Q-Learning Method," in *2021 IEEE Congress on Evolutionary Computation (CEC)*, Kraków, Poland, 2021, pp. 2331–2338.

[52] K. Gurney, *An Introduction to Neural Networks*. Boca Raton, FL, USA: CRC Press, 2018.

[53] G. Reinelt, “TSPLIB, A traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.

[54] J. Schmidhuber, "Deep learning in neural networks: An overview," Neural Networks, vol. 61, pp. 85–117, 2015.

[55] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006.

[56] I. Goodfellow, Y. Bengio, and A. Courville*, Deep Learning*. Cambridge, MA: MIT Press, 2016.

[57] K. Gurney*, An Introduction to Neural Networks*. Boca Raton, FL: CRC Press, 2018.

[58] International Transport Forum, E-Commerce Deliveries and Urban Transport: Facts and Figures. Paris, France: OECD Publishing, 2023.

[59] T. Alam et al., “Genetic Algorithm: Reviews, Implementations, and Applications,” *International Journal of Engineering Pedagogy (iJEP)*, vol. 10, no. 6, pp. 57–77, Dec. 2020, doi: 10.3991/ijep.v10i6.14567.

[60] A. F. Egba and O. R. Okonkwo, “Genetic Algorithm Approach for Optimal Cyclic Tour Round the State Capitals in Nigeria’s Niger Delta Region,” *International Journal of Research - GRANTHAALAYAH*, vol. 9, no. 5, pp. 171–186, May 2021, doi: 10.29121/granthaalayah.v9.i5.2021.3906